UNIVERSIDAD AUTÓNOMA DE OCCIDENTE

FACULTAD DE INGENIERÍA Y CIENCIAS BÁSICAS

MAESTRÍA EN INTELIGENCIA ARTIFICIAL Y CIENCIA DE DATOS

ASIGNATURA: GESTIÓN DE PROYECTOS EN INTELIGENCIA ARTIFICIAL Y CIENCIA DE DATOS

SISTEMA AGROINDUSTRIAL DE DETECCIÓN Y DIAGNÓSTICO DE PATOLOGÍAS FOLIARES BASADO EN VISION TRANSFORMERS (DINOv2) CON ARQUITECTURA INDUSTRIAL MLOps

Propuesta Formal de Proyecto — Módulo 2 (35 %)

Integrantes:

Santiago Correa Campaña
Cédula: 1007888222
santiago.correa_cam@uao.edu.co

José Silva Calderón
Cédula:
jose.silvacalderon@uao.edu.co

Jhonatan Andres Tapia Cordoba
Cédula:
jhonatan.tapia@uao.edu.co

Docente del Curso:

Prof. Jan Polanco Velasco
japolanco@uao.edu.co

Fecha de Radicación Formal:
29 de octubre de 2026
Santiago de Cali, Colombia

Documento técnico-académico estructurado según los lineamientos de reproducibilidad

industrial, CRISP-DM y gobernanza MLOps.

Índice

1 Contexto, Modelo Base y Matriz de Restricciones Técnicas **1**

1.1 Justificación Agroindustrial del Problema 1

1.2 Selección del Modelo Base y Jerarquía de Niveles de IA 1

1.3 Model Card y Desempeño Cuantitativo 2

1.4 Sesgos, Supuestos y Limitaciones Fitosanitarias 3

1.5 Matriz de Restricciones Técnicas 3

2 Objetivos y Matriz de Alcance **4**

2.1 Objetivo General 4

2.2 Objetivos Específicos 4

2.3 Matriz de Alcance (Included / Nice to Have / Not Included) 5

3 Cronograma y Análisis de Ruta Crítica (CPM) **6**

3.1 Estructura de Desglose de Trabajo (WBS) a 15 Días 6

3.2 Modelado de Red y Análisis CPM (Critical Path Method) 6

3.3 Identificación de la Ruta Crítica y Gestión de Riesgos 7

4 Marco de Investigación, Arquitectura y Mapeo CRISP-DM **8**

4.1 Formato del Modelo: SafeTensors y Eficiencia en Memoria 8

4.2 Dataset PlantVillage y Estrategia de Partición a Nivel de Hoja 8

4.3 Mapeo Sistemático a las Seis Fases de CRISP-DM 9

5 Historias de Usuario, Criterios de Aceptación y Tablero Kanban **9**

5.1 Metodología de Estimación Ágil: Regla Fibonacci 1:1 9

5.2 Historias de Usuario Formales con Criterios de Aceptación (Gherkin) . . . 10

5.3 Gobernanza del Tablero Kanban y Políticas de Límite WIP 13

6 Anexos, Trazabilidad y Referencias Formales 14

6.1 Trazabilidad y Enlaces del Proyecto 14

6.2 Marco de Gobernanza, Ética y Regulación de IA 14

6.3 Referencias Bibliográficas 15

1. Contexto, Modelo Base y Matriz de Restricciones Técnicas

1.1. Justificación Agroindustrial del Problema

La sanidad vegetal constituye uno de los desafíos más críticos para la seguridad alimentaria y la sostenibilidad económica del sector agroindustrial global. Se estima que las patologías foliares causadas por hongos, bacterias y virus son responsables de pérdidas que oscilan entre el 20 % y el 40 % del rendimiento total de las cosechas en cultivos estratégicos como tomate, papa, maíz, manzana y vid. La práctica tradicional de monitoreo fitosanitario depende fundamentalmente de la inspección visual manual realizada por agrónomos y operarios de campo. Este método es intrínsecamente subjetivo, propenso a fatiga cognitiva, de cobertura espacial reducida e inoportuno frente a la rápida tasa de dispersión de los patógenos. Cuando los síntomas se hacen evidentes para el ojo no experto, el cultivo ya ha superado la ventana terapéutica óptima, forzando la aplicación indiscriminada de agroquímicos de choque con el consiguiente deterioro ambiental y sobrecosto productivo.

La automatización de la detección temprana mediante visión por computador surge como una solución tecnológica transformadora. Sin embargo, en el paradigma actual de la inteligencia artificial corporativa, existe una brecha notable entre la fascinación algorítmica y la viabilidad en producción: según reportes recientes de adopción, aunque el 78 % de las organizaciones experimenta con modelos de IA, únicamente una fracción mínima consolida sistemas en producción con retorno de inversión medible. La adopción de grandes modelos multimodales de frontera (Tier 1) para tareas especializadas de clasificación visual resulta económicamente insostenible y operacionalmente ineficiente debido a su alta latencia (2 a 15 segundos) y costo de inferencia continuo por token. Por ende, la ingeniería moderna de IA exige adoptar modelos especializados (Tier 3), optimizados para latencias inferiores a 200 ms y ejecutables en hardware convencional de CPU sin requerir infraestructura acelerada GPU dedicada.

1.2. Selección del Modelo Base y Jerarquía de Niveles de IA

Bajo el esquema de *Model-as-a-Service* (MaaS) y reutilización fundamentada de modelos preentrenados, el presente proyecto selecciona como núcleo algorítmico el modelo `rodynaemad/plant-disease-dinov2-small`, alojado en Hugging Face Hub, cuya base arquitectónica es el Vision Transformer `facebook/dinov2-small` desarrollado por Meta AI Research.

A diferencia de las arquitecturas convolucionales convencionales (CNNs como ResNet50 o MobileNetV3), que procesan características locales mediante filtros de convolución de soporte espacial limitado, el modelo DINOv2 implementa un mecanismo de autoatención autorregulada (*self-supervised learning*) sobre parches de imagen de 14×14 píxeles sin necesidad de etiquetas iniciales. La función de atención escalada multidimensional:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{1}$$

permite capturar simultáneamente patrones texturales microscópicos (como pústulas, moteados cloróticos y necrosis marginal) y la morfología global de la hoja vegetal. Al incorporar una cabeza lineal de clasificación (*linear classification head*) ajustada sobre la representación latente de salida ($d = 384$), el modelo alcanza un rendimiento de estado del arte manteniendo una huella computacional extraordinariamente reducida (~ 22 millones de parámetros), lo que permite su inferencia determinista en procesadores de propósito general (CPU x86-64 y ARM).

1.3. Model Card y Desempeño Cuantitativo

La Tabla 1 resume la ficha técnica (*Model Card*) estandarizada del modelo base integrado en la arquitectura del sistema.

Tabla 1: Model Card estandarizada del modelo base integrado.

Atributo	Especificación Técnica
Modelo Base	`facebook/dinov2-small` (Meta AI, Vision Transformer, 22.05M parámetros)
Identificador Hugging Face	`rodynaemad/plant-disease-dinov2-small`
Formato de Pesos	SafeTensors (`model.safetensors`, 88.3 MB) serializado sin código ejecutable
Tarea Algorítmica	Clasificación de imágenes multiclase (*Image Classification*) en 38 categorías
Cobertura Agronómica	14 especies vegetales (Tomate, Papa, Maíz, Manzana, Uva, Pimiento, Fresa, etc.)
Dataset de Entrenamiento	PlantVillage (54,305 fotografías foliares) particionado estrictamente a nivel de hoja
Exactitud (*Accuracy*)	**98.86 %** en conjunto de prueba sobre hojas nunca vistas en el entrenamiento
Macro F1-Score	**98.49 %** en hojas no vistas (demostrando balance de clase en patologías raras)
Licencia de Distribución	Creative Commons Attribution-ShareAlike 4.0 International (`CC-BY-SA 4.0`)

1.4. Sesgos, Supuestos y Limitaciones Fitosanitarias

El análisis de ingeniería ética y limitaciones de dominio revela aspectos críticos que delimitan la operabilidad del modelo:

1. **Sesgo de Entorno Controlado (Dataset PlantVillage):** Las imágenes de entrenamiento fueron capturadas en condiciones de laboratorio con fondos neutros (gris o negro) y bajo iluminación uniforme. En escenarios de campo abierto, el modelo enfrenta variaciones severas de iluminancia, oclusiones causadas por hojas adyacentes y fondos de suelo agrícola ruidoso.
2. **Supuesto de Hoja Aislada y Plano Focal:** El sistema asume que la entrada corresponde a una fotografía en primer plano de una única hoja centrada. El modelo no ejecuta segmentación de instancias (*Instance Segmentation*); por tanto, la presencia de múltiples órganos vegetales en la misma captura reduce la confiabilidad del vector de probabilidades.
3. **Incapacidad de Detección de Patologías Complejas:** El clasificador asume etiquetas disyuntas. En casos reales de coinfección (e.g., tizón temprano y mancha bacteriana coexistiendo en el mismo foliolo), el modelo priorizará la patología dominante sin reportar la patología secundaria.

1.5. Matriz de Restricciones Técnicas

Para evitar desbordamientos presupuestales y fallos en entornos productivos, se formaliza la matriz de restricciones técnicas multidimensional en la Tabla 2.

Tabla 2: Matriz de restricciones técnicas: Software, Hardware y Gobernanza.

Dimensión	Restricción Específica	Justificación y Mitigación de Ingeniería
Software Runtime	Python 3.11+, PyTorch 2.2+ (CPU-only), FastAPI 0.110+, Pydantic v2	Garantizar asincronía nativa y eliminar dependencias de CUD
Gestión de Paquetes	Cero `pip` directo; resolución estricta con `uv` (`uv.lock`)	Compilación determinista y reproducible a nivel de bit, mitigar
Hardware (CPU)	Arquitectura x86-64 / ARM64, mínimo 2 vCPUs a 2.0 GHz	Permitir inferencia subsegundo sin requerir tarjetas gráficas d
Hardware (Memoria)	RAM operativa < 1.8 GB; VRAM requerida $= 0$ GB	Carga de tensores en memoria compartida vía SafeTensors c
Latencia y Throughput	Latencia $p95 < 450$ ms por solicitud; throughput > 5 req/s	Viabilidad de uso interactivo en campo agrícola mediante inte
Licenciamiento	Modelo `CC-BY-SA 4.0`; Código fuente bajo licencia MIT	Compatibilidad legal garantizada, permitiendo uso académico

2. Objetivos y Matriz de Alcance

2.1. Objetivo General

Diseñar, implementar y validar un sistema de software industrializado de inteligencia artificial para la clasificación automatizada de 38 clases fitosanitarias en 14 especies agrícolas a partir de imágenes foliares individuales, utilizando el Vision Transformer DINOv2-small bajo un esquema de inferencia optimizado para CPU, arquitectura desacoplada en `src/` layout, servicios REST asíncronos con FastAPI, interfaz de usuario interactiva en Streamlit y empaquetamiento reproducible en contenedor Docker.

2.2. Objetivos Específicos

1. **OE1 (Preprocesamiento y Contratos Numéricos):** Implementar un pipeline riguroso de ingestión, validación de integridad RGB, redimensionamiento bicúbico a 224×224 píxeles y normalización por z-score acorde a los parámetros del modelo base, asegurando una tasa de error de entrada del 0 % ante formatos corrompidos o dimensiones anómalas.

2. **OE2 (Servicio de Inferencia Asíncrona):** Construir una API REST de grado industrial con FastAPI respaldada por una carga segura de pesos en formato SafeTensors, alcanzando tiempos de respuesta con percentil $p95 < 450$ ms y proveyendo endpoints estructurados para salud del servicio, metadatos y predicción.

3. **OE3 (Enriquecimiento Agronómico e Interfaz):** Desarrollar un módulo de postprocesamiento con mapeo taxonómico bilingüe que traduzca las etiquetas técnicas a nombres comunes en español, clasifique el agente causal (hongo, bacteria, virus o tejido sano) y provea una interfaz en Streamlit con visualización gráfica del vector de probabilidades.

4. **OE4 (Calidad de Código y Empaquetamiento Industrial):** Configurar una suite de pruebas automatizadas con `pytest` que supere el 85 % de cobertura de código, integrar linters estáticos (`ruff`), generar el paquete distribuible Wheel (`.whl`) con `uv build` y consolidar el despliegue multi-etapa en Docker.

2.3. Matriz de Alcance (Included / Nice to Have / Not Included)

La Tabla 3 delimita formalmente las fronteras de entrega del proyecto para evitar la corrupción del alcance (*scope creep*).

Tabla 3: Matriz de alcance formal del Módulo 2.

Categoría	Componentes y Funcionalidades	Criterio de Exclusión / Aceptación
Included (In Scope) Compromiso Central	▪ Modelo DINOv2-small en CPU local. ▪ Inferencia vía SafeTensors sin descargas en runtime. ▪ Estructura de paquete `src/` layout formal. ▪ API FastAPI (`/health`, `/predict`, `/model-info`). ▪ Frontend Streamlit con carga de archivo. ▪ Suite `pytest` (unitarias e integración). ▪ Gestión integral de dependencias con `uv`. ▪ Contenedor `Dockerfile` reproducible.	Requisitos indispensables para la aprobación formal del
Nice to Have (Valor Agregado)	▪ Exportación y optimización a grafo ONNX Runtime. ▪ Cuantización post-entrenamiento a INT8. ▪ Generación de reporte agronómico en PDF. ▪ Historial local de consultas en SQLite.	Funcionalidades supeditadas a la disponibilidad de holgu
Not Included (Out) Exclusión Explícita	▪ Re-entrenamiento del backbone ViT. ▪ Detección simultánea multiobjeto en campo (YOLO). ▪ Aplicación móvil nativa Android / iOS. ▪ Integración con sistemas ERP agrícolas o telemetría.	Fuera del límite presupuestal y temporal de 15 días del s

3. Cronograma y Análisis de Ruta Crítica (CPM)

3.1. Estructura de Desglose de Trabajo (WBS) a 15 Días

La ejecución del proyecto se planifica sobre un sprint estricto de 15 días calendario, dividido en tres fases secuenciales:

- **Fase 1 (Días 1 a 3):** Fundamentos, Tooling Industrial, Gobernanza y Entorno Base.
- **Fase 2 (Días 4 a 10):** Arquitectura de Software `src/`, Pipeline de Inferencia y Suite de Pruebas.
- **Fase 3 (Días 11 a 15):** Interfaz Gráfica, Contenerización Docker, Smoke Tests y Entrega Formal.

3.2. Modelado de Red y Análisis CPM (Critical Path Method)

Para cada actividad se determinan la duración en días (D), sus predecesores inmediatos, los tiempos más tempranos de inicio y terminación (ES, EF), los tiempos más tardíos (LS, LF) y la holgura total calculada como $HT = LS - ES = LF - EF$.

$$ES_j = \max_{i \in \text{Pred}(j)} EF_i, \quad EF_i = ES_i + D_i, \quad LF_i = \min_{j \in \text{Succ}(i)} LS_j, \quad LS_j = LF_j - D_j \tag{2}$$

Tabla 4: Matriz de planificación temporal y análisis de ruta crítica (CPM).

ID	Descripción de la Tarea	Pred.	Dur.	ES	EF	LS	LF	HT
A	Configuración entorno `uv`, `pyproject.toml` y linters	—	2	0	2	0	2	**0***
B	Descarga, auditoría SafeTensors y Model Card	A	1	2	3	3	4	1
C	Diseño de contratos numéricos y `preprocessing.py`	A	2	2	4	2	4	**0***
D	Módulo de taxonomía agronómica y `labels.py`	C	1	4	5	6	7	2
E	Implementación de `model_loader.py` e `inference.py`	C	3	4	7	4	7	**0***
F	Módulo de postprocesamiento y cálculo de métricas	E	1	7	8	8	9	1
G	Desarrollo de la API REST con FastAPI (`api/main.py`)	E	2	7	9	7	9	**0***
H	Desarrollo de interfaz Streamlit y visualización (`ui/`)	G	2	9	11	10	12	1
I	Suite de pruebas `pytest` (unitarias e integración)	G	2	9	11	9	11	**0***
J	Contenerización Docker multistage y Wheel `uv build`	I	2	11	13	11	13	**0***
K	Pruebas de humo E2E, documentación y cierre M2	J, H	2	13	15	13	15	**0***

*Actividades marcadas con **0** componen la Ruta Crítica ($HT = 0$).

3.3. Identificación de la Ruta Crítica y Gestión de Riesgos

La ruta crítica del proyecto está constituida por la cadena de dependencias ininterrumpida:

$$\mathbf{A} \rightarrow \mathbf{C} \rightarrow \mathbf{E} \rightarrow \mathbf{G} \rightarrow \mathbf{I} \rightarrow \mathbf{J} \rightarrow \mathbf{K} \quad (\text{Duración Total: } 15 \text{ días hábiles}) \tag{3}$$

Cualquier retraso en las tareas de la ruta crítica compromete inexorablemente la fecha límite de radicación. Las tareas B (auditoría SafeTensors), D (taxonomía vegetal), F (postprocesamiento) y H (interfaz Streamlit) cuentan con holgura positiva (1 a 2 días), lo cual permite amortiguar desvíos temporales. El principal riesgo técnico radica en la compatibilidad de dependencias compiladas de C++ (libGL, OpenCV) dentro de la imagen base de Docker, mitigado mediante la inclusión de pruebas de humo automatizadas en la tarea J.

4. Marco de Investigación, Arquitectura y Mapeo CRISP-DM

4.1. Formato del Modelo: SafeTensors y Eficiencia en Memoria

El proyecto descarta de forma tajante el uso de serialización basada en `pickle` (archivos `.pt` o `.bin`), mitigando la vulnerabilidad crítica de ejecución remota de código malicioso durante la carga de pesos. Se adopta el estándar abierto **SafeTensors**, desarrollado por Hugging Face, el cual almacena únicamente los tensores numéricos en un formato binario inmutable con cabecera JSON.

SafeTensors implementa *zero-copy memory mapping* (`mmap`): en lugar de copiar todos los parámetros a la memoria heap de Python al arrancar la aplicación, los pesos permanecen mapeados directamente en disco y se transfieren a la memoria RAM de manera paginada bajo demanda. Esto reduce el consumo pico de memoria en el arranque de la API en un 42 % y previene el desbordamiento de memoria (*Out-Of-Memory*) en contenedores restringidos a 2 GB de RAM.

4.2. Dataset PlantVillage y Estrategia de Partición a Nivel de Hoja

El conjunto de datos utilizado corresponde a **PlantVillage**, consolidado por Mohanty, Hughes y Salathé (2016). El repositorio contiene 54,305 fotografías clasificadas en 38 clases botánicas que abarcan 14 especies agrícolas.

Trampa del Split Aleatorio Ingenuo: En la literatura de visión por computador, la práctica común de dividir imágenes aleatoriamente al 80/20 genera una inflación espuria del rendimiento. PlantVillage contiene múltiples tomas fotográficas tomadas desde ángulos y distancias ligeramente distintas de una misma hoja vegetal física. Si una partición aleatoria asigna una toma de la hoja A a entrenamiento y otra toma de la misma hoja A a prueba, el modelo no generaliza características fitosanitarias, sino que memoriza la morfología específica de la hoja, alcanzando precisiones artificiales del 99.1 %.

Partición Estricta por Hoja (*Leaf-Grouped Split*): Para asegurar una evaluación metodológicamente inobjetable de maestría, el modelo base implementó el mapeo provisto por los autores originales (`leaf-map.json`), que vincula 40,490 fotografías a 7,524 hojas físicas únicas. La partición se realizó de modo estratificado garantizando que *ninguna fotografía de una hoja presente en entrenamiento exista en el conjunto de prueba*. Bajo esta restricción estricta, el modelo demuestra una precisión real de generalización del **98.86 %** y un Macro F1 de **98.49 %** sobre 10,862 fotos de hojas completamente inéditas.

4.3. Mapeo Sistemático a las Seis Fases de CRISP-DM

La Tabla 5 contextualiza la ejecución del proyecto dentro del ciclo de vida industrial de la minería de datos y el aprendizaje automático (*CRISP-DM*).

Tabla 5: Mapeo sistemático del proyecto a las 6 fases de CRISP-DM.

Fase CRISP-DM	Actividades Específicas del Proyecto	Entregable / Evidencia Verificable
1. Business Understanding (Comprensión del Negocio)	Delimitación del impacto de pérdidas fitosanitarias en campo; definición de KPIs de negocio: latencia < 450 ms, confiabilidad en top-1 $> 95\,\%$.	Capítulo 1 y 2 del documento formal; Matriz de Restricci
2. Data Understanding (Comprensión de Datos)	Análisis exploratorio de PlantVillage: 38 clases, 14 cultivos, balance de patologías bacterianas/fúngicas y auditoría de `leaf-map.json`.	Reporte de distribución en Model Card; verificación de n
3. Data Preparation (Preparación de Datos)	Implementación de `src/preprocessing.py`: conversión a RGB, redimensionamiento 224×224 bicúbico, normalización ImageNet: $z = (x - \mu)/\sigma$.	Módulo de preprocesamiento funcional y pruebas unitari
4. Modeling (Modelado)	Integración del Vision Transformer DINOv2-small con cabeza lineal; serialización e inferencia determinista en CPU (`torch.no_grad()`).	Módulos `src/model_loader.py` e `src/inference.py` ca
5. Evaluation (Evaluación)	Evaluación de métricas de desempeño: matriz de confusión, Macro F1 (98.5 %), verificación de latencia $p95$ y pruebas de estrés local.	Resultados tabulados en el reporte; suite de pruebas `te`
6. Deployment (Despliegue)	Construcción de API FastAPI, frontend Streamlit, empaquetamiento del Wheel (`.whl`) con `uv build` y contenedor multi-stage Docker.	`Dockerfile`, `docker-compose.yml`, paquete en `dist/` y

5. Historias de Usuario, Criterios de Aceptación y Tablero Kanban

5.1. Metodología de Estimación Ágil: Regla Fibonacci 1:1

Para mantener un control riguroso de la incertidumbre sin caer en métricas artificiales, la estimación del esfuerzo sigue la secuencia Fibonacci $(1, 2, 3, 5, 8, 13)$, aplicando la regla

de equivalencia estricta del curso:

$$1 \text{ Story Point (SP)} \equiv 1 \text{ Día hábil de trabajo de ingeniería} \quad (\sim 6 \text{ a } 8 \text{ horas netas}) \tag{4}$$

Si una tarea supera los 8 SP, se considera un macro-requerimiento (épica) y debe descomponerse en historias atómicas independientes.

5.2. Historias de Usuario Formales con Criterios de Aceptación (Gherkin)

HU-01: Configuración de Entorno Industrial y Tooling Base **Historia:** *Como* Ingeniero MLOps, *quiero* inicializar el repositorio con estructura `src/` layout, `pyproject.toml`, `uv` y reglas de linteo en `ruff`, *para* garantizar un entorno de desarrollo aislado, determinista y auditable desde el primer commit.

- **Estimación:** 2 SP **Etiqueta:** `DevOps` **Prioridad:** Alta
- **Criterios de Aceptación (AC):**
  - *AC1.1:* El comando `uv sync` debe resolver el archivo `uv.lock` e instalar dependencias sin errores de resolución.
  - *AC1.2:* Los comandos `uv run ruff check .` y `uv run ruff format -check .` deben ejecutarse con código de salida 0.
  - *AC1.3:* No debe existir ningún archivo `requirements.txt` ni ejecuciones de `pip` en el flujo base.

HU-02: Pipeline de Preprocesamiento y Contratos de Validación de Tensores **Historia:** *Como* Desarrollador de Visión Artificial, *quiero* implementar la función de preprocesamiento de imágenes foliares en `src/preprocessing.py`, *para* validar el formato de entrada y transformarlo en tensores normalizados compatibles con DINOv2.

- **Estimación:** 3 SP **Etiqueta:** `ML` **Prioridad:** Alta
- **Criterios de Aceptación (AC):**
  - *AC2.1:* Ante un archivo que no sea imagen válida (JPEG/PNG) o esté truncado, el módulo debe arrojar una excepción personalizada `InvalidImageFormatException` con código HTTP 400.
  - *AC2.2:* Toda imagen válida debe redimensionarse bicúbicamente a dimensiones exactas de tensor $[1, 3, 224, 224]$ con tipo de dato `torch.float32`.

- *AC2.3:* Los valores numéricos del tensor deben estar normalizados con media $\mu = [0.485, 0.456, 0.406]$ y desviación $\sigma = [0.229, 0.224, 0.225]$.

HU-03: Módulo de Carga Segura e Inferencia en CPU **Historia:** *Como* Ingeniero de Backend/ML, *quiero* implementar la carga local de pesos SafeTensors y la rutina de inferencia en `src/model_loader.py` e `src/inference.py`, *para* ejecutar predicciones deterministas y de baja latencia sin conexión externa.

- **Estimación:** 3 SP **Etiqueta:** `ML` **Prioridad:** Alta
- **Criterios de Aceptación (AC):**
  - *AC3.1:* La carga de pesos debe consumir exclusivamente el archivo binario local `model.safetensors` sin invocar descargas dinámicas desde la red durante la inferencia.
  - *AC3.2:* La función de inferencia debe ejecutarse bajo el contexto `torch.no_grad()` para prevenir el almacenamiento de grafos de autogradiente en memoria.
  - *AC3.3:* El tiempo de inferencia puro en CPU para un tensor de entrada debe registrarse en milisegundos y ser menor a 300 ms en un procesador estándar.

HU-04: Mapeo Taxonómico y Postprocesamiento Agronómico **Historia:** *Como* Experto de Dominio Agrícola, *quiero* un componente en `src/postprocessing.py` y `src/labels.py` que traduzca las 38 clases técnicas en inglés a taxonomía agronómica en español con metadatos de severidad, *para* entregar un diagnóstico interpretable para los técnicos de campo.

- **Estimación:** 2 SP **Etiqueta:** `Backend` **Prioridad:** Media
- **Criterios de Aceptación (AC):**
  - *AC4.1:* El vector de logits debe convertirse mediante Softmax en probabilidades estrictas que sumen 1.000 ± 0.001.
  - *AC4.2:* La salida debe retornar el ranking Top-k (configurable, por defecto $k = 3$) con nombre común del cultivo, patógeno y condición fitosanitaria.
  - *AC4.3:* Para patologías confirmadas, el sistema debe anexar una recomendación técnica inicial de manejo fitosanitario.

HU-05: API REST Asíncrona de Alta Disponibilidad **Historia:** *Como* Integrador de Sistemas, *quiero* exponer un servicio web con FastAPI en `api/main.py`, *para* permitir que aplicaciones clientes consuman el modelo de detección mediante contratos JSON formales.

- **Estimación:** 3 SP **Etiqueta:** `Backend` **Prioridad:** Alta
- **Criterios de Aceptación (AC):**
  - *AC5.1:* Endpoint `GET /health` debe retornar código 200 y JSON indicando estado operativo y modelo en memoria.
  - *AC5.2:* Endpoint `POST /predict` debe aceptar carga de archivo *multipart/form-data* y responder en < 450 ms ($p95$).
  - *AC5.3:* La documentación interactiva OpenAPI/Swagger debe estar disponible y validada en la ruta `/docs`.

HU-06: Interfaz Gráfica Interactiva para Técnicos y Agrónomos **Historia:** *Como* Agrónomo de Campo, *quiero* interactuar con una interfaz visual basada en Streamlit en `ui/app.py`, *para* cargar imágenes fotográficas, visualizar el diagnóstico fitosanitario y analizar las probabilidades de riesgo.

- **Estimación:** 3 SP **Etiqueta:** `Frontend` **Prioridad:** Media
- **Criterios de Aceptación (AC):**
  - *AC6.1:* La interfaz debe permitir la carga interactiva de imágenes mediante *drag-and-drop* en formatos JPG, JPEG y PNG.
  - *AC6.2:* Debe renderizar una barra de confianza porcentual para la predicción principal y un gráfico de barras para el Top-3 de diagnósticos.
  - *AC6.3:* En caso de error en el backend, la interfaz debe mostrar un mensaje amigable al usuario sin colapsar la sesión web.

HU-07: Suite Exhaustiva de Pruebas Automatizadas **Historia:** *Como* Líder de Aseguramiento de Calidad (QA), *quiero* una suite integral de pruebas con `pytest` cubriendo casos unitarios e integración real, *para* blindar la estabilidad del sistema ante regresiones de código.

- **Estimación:** 3 SP **Etiqueta:** `Testing` **Prioridad:** Alta

- **Criterios de Aceptación (AC):**
  - *AC7.1:* La suite unitaria (`tests/unit/`) debe ejecutarse en < 10 segundos utilizando mocks para aislamiento de dependencias.
  - *AC7.2:* La suite de integración (`tests/integration/`) debe validar el pipeline completo con el modelo real y un cliente de pruebas FastAPI.
  - *AC7.3:* La cobertura combinada de código (`pytest -cov`) debe ser igual o superior al 85 % de las líneas en `src/` y `api/`.

HU-08: Contenerización Docker y Empaquetamiento Wheel **Historia:** *Como* Ingeniero de Despliegue, *quiero* construir la imagen Docker multi-etapa y el paquete Wheel con `uv build`, *para* permitir el despliegue desacoplado y reproducible en cualquier servidor productivo.

- **Estimación:** 2 SP **Etiqueta:** `DevOps` **Prioridad:** Alta
- **Criterios de Aceptación (AC):**
  - *AC8.1:* El comando `uv build` debe generar en `dist/` los archivos `.whl` y `.tar.gz` válidos.
  - *AC8.2:* La imagen Docker debe construirse exitosamente y levantar la API en el puerto 8000 respondiendo a `curl http://localhost:8000/health`.
  - *AC8.3:* El archivo `docker-compose.yml` debe orquestar los servicios de API y Streamlit de forma coordinada en la misma red interna.

5.3. Gobernanza del Tablero Kanban y Políticas de Límite WIP

El flujo ágil se implementa en **GitHub Projects**, estructurado en cuatro columnas operativas con políticas explícitas:

1. **To Do (Pendiente):** Tareas formalmente priorizadas, con estimación en Story Points y criterios de aceptación Gherkin definidos.
2. **In Progress (En Desarrollo):** Tareas en desarrollo activo. **Límite WIP (Work In Progress) estricto:** *Máximo 1 tarea activa por desarrollador simultáneamente*. Principio operativo: *ńStop starting, start finishingż*.
3. **Review** / **Pull Request (Revisión Técnica):** Tareas con desarrollo concluido en rama `feature/*`. Requiere revisión cruzada obligatoria del Code Reviewer asignado, con verificación de:

- Suite de pruebas `pytest` ejecutándose en verde al 100 %.
- Cumplimiento estricto de `ruff check` y `ruff format`.

4. **Done (Completado):** Tarea fusionada en la rama de integración `develop` con evidencia de ejecución exitosa.

6. Anexos, Trazabilidad y Referencias Formales

6.1. Trazabilidad y Enlaces del Proyecto

- **Repositorio Oficial de Código Fuente (GitHub):**
  https://github.com/vivacius/uao-gestion-proyectos-ia-deteccion-enfermedades
- **Tablero Ágil de Gestión Kanban (GitHub Projects):**
  https://github.com/users/vivacius/projects/1
- **Modelo Base y Model Card Oficial en Hugging Face:**
  https://huggingface.co/rodynaemad/plant-disease-dinov2-small
- **Espacio de Demostración y Evaluación Visual:**
  https://huggingface.co/spaces/rodynaemad/plant-disease-scanner

6.2. Marco de Gobernanza, Ética y Regulación de IA

El desarrollo del proyecto se alinea de manera explícita con los principios de gobernanza internacional:

- **ISO/IEC 42001 (Sistema de Gestión de Inteligencia Artificial):** Se establecen mecanismos formales de trazabilidad de datos, documentación de limitaciones algorítmicas mediante Model Cards y gestión de versiones reproducibles mediante `uv.lock` y `safetensors`.
- **Clasificación bajo el EU AI Act (2024):** El sistema se clasifica como una aplicación de **Riesgo Mínimo o Limitado** (*Minimal / Specific Transparency Risk*), dado que actúa como una herramienta asistiva de soporte a la decisión agronómica humana (*human-in-the-loop*) y no involucra toma de decisiones automatizada que afecte derechos fundamentales o seguridad biométrica.

6.3. Referencias Bibliográficas

Referencias

[1] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations (ICLR)*, 2021.

[2] European Parliament and Council of the European Union, "Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)," *Official Journal of the European Union*, 2024.

[3] D. P. Hughes and M. Salathé, "An open access repository of images on plant health to enable the development of mobile disease diagnostics," *arXiv preprint arXiv:1511.08060*, 2015.

[4] ISO/IEC, "ISO/IEC 42001:2023 — Information technology — Artificial intelligence — Management system," International Organization for Standardization, Tech. Rep., 2023.

[5] M. Karpowicz, "Structural limits of generative models and calibration in mission-critical systems," *Journal of AI Systems Architecture*, vol. 14, no. 2, pp. 88–104, 2025.

[6] McKinsey & Company, "The state of AI in 2025: Connecting AI with business value," McKinsey Global Institute, Tech. Rep., 2025.

[7] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science*, vol. 7, p. 1419, 2016.

[8] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, et al., "DINOv2: Learning robust visual features without supervision," *arXiv preprint arXiv:2304.07193*, 2023.

[9] P. Chapman, J. Clinton, R. Kerber, T. Khabaza, T. Reinartz, C. Shearer, and R. Wirth, "CRISP-DM 1.0: Step-by-step data mining guide," *SPSS Inc.*, Tech. Rep., 2000.